

Embeddings — Lecture Session

The conceptual deep-dive: how language models represent meaning as numbers *The foundation that powers every language model — including the tiny SLM you will build.*

Course repository: [tri-AI-Teaching](#) Lecture deck: [embeddings.html](#) Hands-on notebook (Colab): [Embeddings Lab](#)

0. Why This Session Matters

Every modern language model — from a tiny classroom SLM to GPT-4 — shares one core idea: **it represents words as lists of numbers called embeddings**. Before we can build or train anything, we need to understand what embeddings are, how similarity is measured, and how to *see* them. This lecture covers exactly that, conceptually and with live examples.

Roadmap: Next, in the lab/SLM session, we take these same vectors and *learn* them from data so a model can predict the next word.

1. What Is an Embedding?

A computer only understands numbers. So a language model turns every word (token) into a long list of numbers called a **vector**. We call that vector the word's **embedding**.

- **A Vector = Coordinates** — Think of an embedding as a point in a giant "meaning space." Each number is one coordinate of that point.
- **Many Numbers** — Big models like **Gemma** use **1,152 numbers** per word. Our tiny SLM used just 32. More numbers = more room for meaning.
- **Similar = Nearby** — Words with similar meaning get *similar* vectors — they sit close together in that space.

```
Embedding for "king" → [0.23, -1.45, 0.67, ..., 0.12] (1,152 numbers)
Embedding for "queen" → [0.25, -1.40, 0.62, ..., 0.15] (almost the same!)
Embedding for "apple" → [-0.98, 2.10, -0.33, ..., 0.88] (very different)
```

Quick check: What is a word embedding? **A)** A table that stores all the sentences a model has read **B)** A long list of numbers that represents a word's meaning **C)** A programming language used to train models **D)** The raw text before it is cleaned

2. Vectors & Matrices (with NumPy)

To work with embeddings we use the Python library `numpy`. It makes working with lists of numbers (vectors) and tables of numbers (matrices) easy.

- **Vector** — An ordered list of numbers. Example: `v = [7, 3, 1, 4]` is a 4-number vector.
- **Matrix** — A table of numbers with rows and columns. The embedding table has *one row per word*.
- **Shape** — How big it is. A vector of 4 numbers has shape `(4,)`. A 2x3 matrix has shape `(2, 3)`.

```
import numpy as np

# A vector and a 2x3 matrix
v = np.array([7, 3, 1, 4])
M = np.array([[6, 1, 4],
              [9, 0, 2]])

print(v.shape)  # (4,) -> 4 numbers
print(M.shape)  # (2, 3) -> 2 rows, 3 columns

# Grab one row (one word's embedding) and one column
row_3 = embeddings[2, :]  # 3rd word's vector
col_7 = embeddings[:, 6]  # 7th number across all words
```

The whole embedding table is just a big matrix: **rows = words, columns = meaning-coordinates**.

Quick check: In the embedding matrix, what does one *row* represent? **A)** One meaning-coordinate shared by all words **B)** The entire model **C)** The embedding (vector) of a single word **D)** A single training sentence

3. Meaning Lives in Directions

The magic of embeddings: words that mean similar things point in **similar directions**. "king" and "queen" point almost the same way. "apple" points somewhere else entirely.

(A simplified 2D map)

king • queen • prince • apple • banana •
phone • computer •
bus •

- Words near each other (royals, fruits, tech) form natural clusters.
- This is a simplified 2D view — in reality the space has **1,152 dimensions**.

This is why a model can **guess meaning** even for words it has rarely seen — their vectors land near related words it already knows.

Quick check: If "cat" and "dog" have very similar embeddings, what does that tell us? **A)** They are spelled the same way **B)** The model thinks they are related in meaning **C)** They appear the same number of times in the data **D)** The embeddings are broken

4. The Dot Product: Measuring Similarity

How do we measure if two vectors point the same way? With the **dot product** — we multiply their numbers pair-by-pair and add everything up.

$$\text{dot_product} = \mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + \dots + u_nv_n$$

Dot product	Angle	Meaning
Positive (> 0)	$< 90^\circ$	Vectors point the <i>same way</i> → words are related
Zero (≈ 0)	$= 90^\circ$	Vectors are <i>unrelated</i> (orthogonal)
Negative (< 0)	$> 90^\circ$	Vectors point <i>opposite ways</i> → words are dissimilar

$\mathbf{u} \cdot \mathbf{v} > 0 \rightarrow$ similar meaning (king · queen)
 $\mathbf{u} \cdot \mathbf{v} = 0 \rightarrow$ no relation (king · bus)
 $\mathbf{u} \cdot \mathbf{v} < 0 \rightarrow$ opposite meaning (hot · cold)

A dot product can grow very large in high dimensions, so we usually **normalize** it — that leads us to cosine similarity.

Quick check: A dot product of **0** between two word embeddings means the words are... **A)** Exactly the same word **B)** Opposites in meaning **C)** Unrelated (at a right angle in the space) **D)** Both very common words

5. Cosine Similarity

Cosine similarity is the **dot product divided by the lengths** of both vectors. The result is always between **-1 and +1**, so it is easy to compare.

$$\text{cosine}(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v}) / (||\mathbf{u}|| \times ||\mathbf{v}||)$$

Pair of words	Cosine	What it means
"king" & "king"	+1.00	Identical
"king" & "queen"	+0.42	Closely related
"joy" & "happy"	high +	Similar feeling
"sad" & "happy"	+0.22	Both describe feelings
"king" & "bus"	+0.04	Unrelated

- **+1** = same direction, **0** = unrelated, **-1** = opposite.
- Notice even antonyms like "sad/happy" score above 0 — they are still both *feelings*, so the model groups them loosely.

Quick check: "king" and "bus" have a cosine similarity of **0.04**. This means they are... **A)** Synonyms **B)** Unrelated **C)** Opposites **D)** The same word

6. One Number Isn't Enough

It is tempting to ask "what does the 5th number *mean*?" In practice, **no single number carries a clear meaning**. Meaning comes from the whole vector together.

- **Hard to Read** — If you plot just two of the 1,152 numbers, words rarely form clean groups. Dimensions mix many ideas at once.
- **Combinations Matter** — Meaning lives in the *pattern across all numbers*, like a recipe — one ingredient alone isn't the dish.
- **So We Reduce** — We use tricks to *compress* all dimensions into 2 or 3 so we can actually see the structure.

```
# Pick any 2 dimensions and plot the words
emb.plot_embeddings_dimensions(embeddings, labels,
                              dim_x=432, dim_y=877)
```

Try different pairs — you will rarely find a single pair that cleanly separates "food" from "technology". That is normal!

Quick check: Why can't we understand an embedding by looking at just one of its numbers? **A)** The numbers are randomly generated every time **B)** Meaning comes from the combination of *all* the numbers together **C)** Embeddings have no real meaning **D)** The computer is too slow to show one number

7. Seeing in 2D: t-SNE

t-SNE is a technique that compresses 1,152 dimensions down to 2, while trying to keep nearby words nearby and far words far. The result is a map we can actually look at.

```
emb.plot_embeddings_tsne(embeddings, labels)
```

- **Clusters Appear** — Related words form tight clouds: "mobile", "phone", "computer", "internet" sit together.
- **Royals Together** — "man"/"woman" and "king"/"queen" land close — the model learned the relationship.
- **Some Noise** — Compression isn't perfect: "cat" and "car" may look near each other by accident.

t-SNE is a **viewing tool**, not the model itself. It helps us *see* and debug what the embeddings learned.

Quick check: What is the main job of t-SNE in this lab? **A)** To make the embeddings bigger **B)** To project high-dimensional embeddings into 2D so we can see clusters **C)** To delete rare words **D)** To speed up training

8. Try It Yourself (Live)

In the hands-on part we run the Google DeepMind notebook and watch embeddings come alive. You will:

1. Load 24 real token embeddings from the **Gemma** model.
2. Write a tiny function to pull out one word's vector.
3. Compute **cosine similarity** between word pairs like "king"/"queen".
4. Plot single dimensions, then use **t-SNE** to see the clusters.

Notebook: [Open the Embeddings Lab notebook in Colab](#) *Original source: Google DeepMind · AI Foundations · course_2*

No answers are hidden — run the cells, change the words, and **discuss what you see**. That is where the learning happens!

Discussion prompt: Which pair do you *predict* will have the highest cosine similarity?

- A) "king" & "queen"
- B) "king" & "bus"
- C) "apple" & "computer"
- D) "water" & "mountain"

9. Key Takeaways

1. **Words → Vectors** — An embedding is a list of numbers that captures a word's meaning. Similar words → similar vectors.
2. **Similarity** — The **dot product** and **cosine similarity** measure how alike two words are (−1 to +1).
3. **See the Space** — Single numbers don't explain meaning; **t-SNE** lets us visualize the whole space as clusters.

Today you **experimented** with real embeddings — the same idea that powers the "meaning" inside every language model, including the tiny SLM we build in the lab session.

10. Resources

- **Lecture deck:** [embeddings.html](#)
- **Hands-on notebook:** [Embeddings Lab \(Colab\)](#)
- **Resources page:** [embeddings-resources.html](#)

- **Course repo:** github.com/AlexKalil/tri-AI-Teaching
- **Interactive playgrounds:**
 - [Tokenizer Playground](#)
 - [Word Embedding Playground](#)
 - [Embeddings Playground \(sheth.io\)](#)
-

Instructor: Kaletsidik Ayalew **GitHub:** [AlexKalil](#) · **LinkedIn:** [kaletsidik-ayalew](#)